First Move Robotics

Software Documentation

Version 1.0.0

Strydr Silverberg
strydr_silverberg@mines.edu

February 12, 2026

A high-performance chess engine written in C++

Contents

1 Introduction **4**
- 1.1 Overview . . . 4
- 1.2 Features . . . 4
- 1.3 System Requirements . . . 4

2 Installation **5**
- 2.1 Building from Source . . . 5
- 2.2 Pre-built Binaries . . . 5
- 2.3 Configuration . . . 5

3 Architecture **6**
- 3.1 Overview . . . 6
- 3.2 Board Representation . . . 6
  - 3.2.1 Data Structures . . . 6
  - 3.2.2 Chess Position Example . . . 6
- 3.3 Move Generation . . . 7
  - 3.3.1 Legal Move Generation . . . 7

4 Search Algorithm **8**
- 4.1 Overview . . . 8
- 4.2 Alpha-Beta Search . . . 8
- 4.3 Search Enhancements . . . 8
  - 4.3.1 Transposition Table . . . 8
  - 4.3.2 Move Ordering . . . 8
  - 4.3.3 Pruning Techniques . . . 9

5 Evaluation **10**
- 5.1 Overview . . . 10
- 5.2 Material . . . 10
- 5.3 Positional Factors . . . 10
  - 5.3.1 Piece-Square Tables . . . 10
- 5.4 King Safety . . . 10
- 5.5 Pawn Structure . . . 10

6 UCI Protocol **11**
- 6.1 Overview . . . 11
- 6.2 Supported Commands . . . 11
  - 6.2.1 Standard UCI Commands . . . 11
  - 6.2.2 Engine Options . . . 11

7 API Reference **12**
- 7.1 Core Classes . . . 12
  - 7.1.1 Position . . . 12

7.1.2 Search 12
7.2 Utility Functions 12

8 Testing **13**
8.1 Perft Testing 13
8.2 Test Positions 13

9 Performance **14**
9.1 Benchmarks 14
9.2 Optimization Techniques 14

10 Contributing **15**
10.1 Development Setup 15
10.2 Code Style 15
10.3 Pull Request Process 15

A Glossary **16**

B References **17**

Chapter 1

Introduction

1.1 Overview

This document provides comprehensive documentation for the **Kirin Chess Engine**. Kirin is a chess engine written in C++ and an integral part of the **First Move Robotics** chess board project.

1.2 Features

- Feature 1: Brief description
- Feature 2: Brief description
- Feature 3: Brief description

1.3 System Requirements

Component	**Requirement**
Operating System	Windows / Linux / macOS
Memory	Minimum 256 MB RAM
Processor	Any modern x86-64 CPU
Dependencies	List dependencies here

Table 1.1: Minimum system requirements

Chapter 2

Installation

2.1 Building from Source

Clone the repository and build:

```
git clone https://github.com/username/chess-engine.git
cd chess-engine
make
```

2.2 Pre-built Binaries

Download the latest release from the releases page.

2.3 Configuration

The engine can be configured via the `config.ini` file:

```
[Engine]
hash_size = 256
threads = 4
ponder = true
```

Chapter 3

Architecture

3.1 Overview

Describe the high-level architecture of your engine here.

3.2 Board Representation

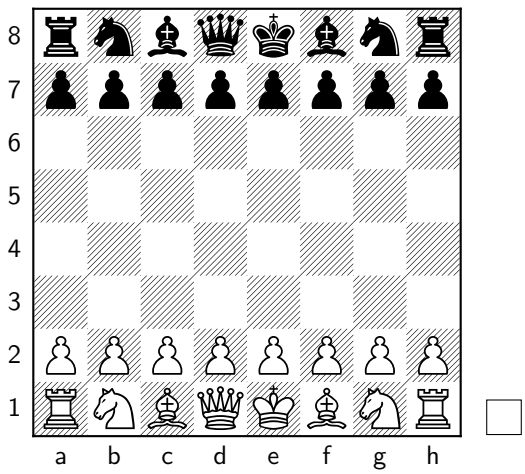
3.2.1 Data Structures

Describe how the board is represented (bitboards, mailbox, 0x88, etc.).

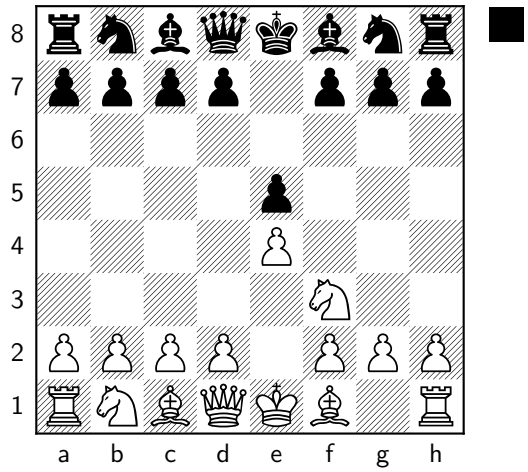
```
// Example bitboard representation
struct Position {
    uint64_t pieces[2][6];  // [color][piece_type]
    uint64_t occupied[2];   // [color]
    uint64_t all_pieces;
    int side_to_move;
    int castling_rights;
    int en_passant_square;
    int halfmove_clock;
    int fullmove_number;
};
```

3.2.2 Chess Position Example

Here is an example starting position:



After `1.e4 e5 2.Nf3`:



3.3 Move Generation

Describe your move generation approach.

3.3.1 Legal Move Generation

Algorithm 1 Legal Move Generation

```
procedure GenerateMoves(position)
    moves ← empty list
    for each piece of side to move do
        Generate pseudo-legal moves for piece
        for each pseudo-legal move do
            if move does not leave king in check then
                Add move to moves
            end if
        end for
    end for
    return moves
end procedure
```

Chapter 4

Search Algorithm

4.1 Overview

Describe your search algorithm (Alpha-Beta, MCTS, etc.).

4.2 Alpha-Beta Search

Algorithm 2 Alpha-Beta with Negamax

```
function AlphaBeta(position, depth, α, β)
    if depth = 0 then
        return Evaluate(position)
    end if
    for each move in GenerateMoves(position) do
        MakeMove(position, move)
        score ← −AlphaBeta(position, depth−1, −β, −α)
        UnmakeMove(position, move)
        if score ≥ β then
            return β                                    ▷ Beta cutoff
        end if
        if score > α then
            α ← score
        end if
    end for
    return α
end function
```

4.3 Search Enhancements

4.3.1 Transposition Table

Describe your transposition table implementation.

4.3.2 Move Ordering

1. Hash move (from transposition table)
2. Winning captures (MVV-LVA)
3. Killer moves
4. History heuristic
5. Quiet moves

4.3.3 Pruning Techniques

- Null Move Pruning
- Late Move Reductions (LMR)
- Futility Pruning
- Delta Pruning (in quiescence)

Chapter 5

Evaluation

5.1 Overview

Describe your evaluation function approach.

5.2 Material

Piece	Value (centipawns)
Pawn	100
Knight	320
Bishop	330
Rook	500
Queen	900

Table 5.1: Piece values

5.3 Positional Factors

5.3.1 Piece-Square Tables

```
// Example: Pawn piece-square table (middlegame)
const int pawn_psqt[64] = {
     0,  0,  0,  0,  0,  0,  0,  0,
    50, 50, 50, 50, 50, 50, 50, 50,
    10, 10, 20, 30, 30, 20, 10, 10,
     5,  5, 10, 25, 25, 10,  5,  5,
     0,  0,  0, 20, 20,  0,  0,  0,
     5, -5,-10,  0,  0,-10, -5,  5,
     5, 10, 10,-20,-20, 10, 10,  5,
     0,  0,  0,  0,  0,  0,  0,  0
};
```

5.4 King Safety

Describe king safety evaluation.

5.5 Pawn Structure

Describe pawn structure evaluation: doubled pawns, isolated pawns, passed pawns, etc.

Chapter 6

UCI Protocol

6.1 Overview

The engine communicates via the Universal Chess Interface (UCI) protocol.

6.2 Supported Commands

6.2.1 Standard UCI Commands

IN `uci`
Initialize UCI mode and request engine identification.

IN `isready`
Check if engine is ready.

IN `position [fen | startpos] moves ...`
Set up a position.

IN `go [options]`
Start calculating. Options include:

- `depth n` – search to depth n
- `movetime ms` – search for ms milliseconds
- `wtime/btime ms` – time remaining
- `infinite` – search until stopped

IN `stop`
Stop calculating.

IN `quit`
Exit the engine.

6.2.2 Engine Options

```
setoption name Hash value 256
setoption name Threads value 4
setoption name Ponder value true
```

Chapter 7

API Reference

7.1 Core Classes

7.1.1 Position

`Position()` *Constructor*
Parameters: None
Returns: New Position object

`set_fen(fen: string)` *Method*
Parameters: fen – FEN string
Returns: bool – success

`make_move(move: Move)` *Method*
Parameters: move – Move to make
Returns: bool – success

7.1.2 Search

`search(position: Position, limits: SearchLimits)` *Function*
Parameters: position, limits
Returns: SearchResult

7.2 Utility Functions

`perft(position: Position, depth: int)` *Function*
Parameters: position, depth
Returns: uint64 – node count

Chapter 8

Testing

8.1 Perft Testing

Perft (performance test) validates move generation:

```
./engine perft 6
```

Depth	Nodes
1	20
2	400
3	8,902
4	197,281
5	4,865,609
6	119,060,324

Table 8.1: Perft results from starting position

8.2 Test Positions

Include strategic and tactical test suites (WAC, ECM, STS, etc.).

Chapter 9

Performance

9.1 Benchmarks

Document performance metrics here.

9.2 Optimization Techniques

- Magic bitboards for sliding piece attacks
- SIMD instructions where applicable
- Cache-friendly data structures
- Efficient hash table implementation

Chapter 10

Contributing

10.1 Development Setup

Instructions for setting up a development environment.

10.2 Code Style

Document your code style guidelines.

10.3 Pull Request Process

1. Fork the repository
2. Create a feature branch
3. Make changes and add tests
4. Submit a pull request

Appendix A

Glossary

Alpha-Beta A search algorithm that prunes branches that cannot affect the final result.

Bitboard A 64-bit integer representing piece positions on the board.

Centipawn One hundredth of a pawn; standard unit for evaluation scores.

FEN Forsyth-Edwards Notation; a standard for describing chess positions.

Perft Performance test; counts all leaf nodes at a given depth.

Ply A half-move; one move by either white or black.

UCI Universal Chess Interface; standard protocol for chess engines.

Appendix B

References

- Chess Programming Wiki: https://www.chessprogramming.org/
- UCI Protocol Specification
- Stockfish source code: https://github.com/official-stockfish/Stockfish